Aula 08 – Subprogramas
Exercício: qual é a saída?

Aluno: Gabriel Santos Furlanette RA: 24000873-2

Disciplina: Paradigmas de Linguagens de Programação

Para cada trecho: (a) qual é a saída? (b) qual conceito da aula explica?

1) Python

```
def adicionar(item, lista=[]):
    lista.append(item)
    return lista

print(adicionar(1))
print(adicionar(2))
```

a) [1] e depois [1, 2]

b) A lista padrão `lista=[]` é criada uma vez só, quando a função é definida, e não a cada chamada. Então a segunda chamada usa a mesma lista que já tinha o 1. Tem a ver com parâmetro com valor padrão. O certo seria usar `lista=None` e criar a lista dentro da função.

2) Java

```
static void zera(int[] v, int n) {
    v[0] = 0;
    n = 0;
}
int[] v = {5, 5}; int n = 5;
zera(v, n);
System.out.println(v[0] + " " + n);
```

a) 0 5

b) Java sempre passa por valor. O `n` é uma cópia, então mudar ele dentro do método não muda o de fora. Já o `v` é uma cópia da referência, mas aponta pro mesmo array, por isso o `v[0]` muda.

3) Python

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

a) [2, 2, 2]

b) Closure. As lambdas guardam a variável `i`, não o valor dela no momento. Quando elas são chamadas o laço já acabou e o `i` vale 2. Dá pra corrigir com `lambda i=i: i`.

4) C

```
int contador(void) {
    static int n = 0;
    return ++n;
}
// em main:
contador(); contador();
printf("%d\n", contador());
```

a) 3

b) Variável local `static`. Ela é inicializada só uma vez e mantém o valor entre as chamadas, então vai 1, 2 e 3. Sem o static ia imprimir 1.

5) Rust

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}
let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?} {:?}", v, d);
```

a) Não compila (erro: borrow of moved value: v)

b) Passagem de parâmetro com ownership. Quando passa `v` pra função ele é movido, e depois não dá pra usar no println. Corrigindo pra receber `&Vec<i32>` e chamar `dobra(&v)`, imprime [1, 2, 3] [2, 4, 6].

6) Python

```
total = 0

def adiciona(x):
    total = total + x
    return total

print(adiciona(5))
```

a) Dá erro: UnboundLocalError

b) Escopo. Como tem uma atribuição pra `total` dentro da função, o Python considera ela local, e aí tenta ler antes de ter valor. Pra usar a de fora tem que colocar `global total` no começo da função.